AI-Native Engineer Capstone

PayBridge — Payments Routing, Reconciliation & Settlement Hub

Business Case BC-AINE-009 · Domain: BFS — Payments Operations

1. Project Identity

Business Case Title	Payments Routing, Reconciliation & Settlement Hub
Business Case ID	BC-AINE-009
Domain	BFS — Payments Operations
Project Type	AI-Native Engineering Capstone
Cohort	(filled by Operations team)

2. Engagement Overview

Duration	20 hours
Format	Individual
Evaluation Mode	Automated review of submitted Git repository against the AI-Native Engineering Rubric (21 parameters / 100 marks across 5 categories)
Submission	Push final code to your assigned Virtusa GitLab repository by the cohort cut-off
Review Output	Per-learner Excel report (Summary, Categories, Scorecard, Detailed, Improvement)
Grade Bands	Distinction ≥ 90 · Merit 75–89 · Pass 60–74 · Not Yet Passed < 60

What is evaluated: AI-native engineering practices (specs, agents, skills, hooks, CLAUDE.md hierarchy, plugin packaging, MCP, Harness plugin usage), business understanding (context, rules, AC), architectural discipline (structural rules, TDD), technical implementation (code, tests, coverage, UI validation), and CI/CD integration (PR-driven workflow, Claude in CI).

What is not evaluated: the specific technology stack chosen, the visual polish of the UI, or the use of any AI provider beyond Claude Code.

3. Problem Statement & Expected Solution

3.1 Problem

A bank wants to consolidate its multiple payment rails (UPI, NEFT, RTGS, IMPS) behind a single hub that handles routing, idempotency, reconciliation, refunds, and daily settlement. You build this hub — under one constraint: no manual coding. All external rails are stubbed adapters with configurable

success/failure responses; every line of production code must be generated by Claude Code agents under your supervision.

3.2 Your Role

AI-Native Engineer — architect, supervisor, and quality gatekeeper. You write specifications, configure agents, install guardrails, and review pull requests. You do not write production code by hand.

3.3 Expected Solution

A working platform delivered as a Git repository that demonstrates:

- A complete AI-native engineering substrate built on the Claude Harness Engine plugin foundation, extended with payments-operations-specific specs, agents, skills, hooks, commands, and MCP integration.
- A functional payments hub built end-to-end by Claude Code agents — covering rail routing, idempotency, retry handling, reconciliation, refund flows, and daily settlement file generation — with no hand-written production code.
- An evidence trail (PRs, agent logs, post-mortems, sprint contracts) proving the work was agent-led.

3.4 Applicable Rules

- **No-Hand-Coding Rule.** All production code, tests, and migrations must be generated by Claude Code agents. Hand-edits are limited to specs, CLAUDE.md, agent definitions, hooks, skills, and other substrate files.
- **Spec-Is-Truth Rule.** Whenever a spec and code disagree, the spec wins. Update the spec deliberately and regenerate the code.
- **PR-Only Merge Rule.** No direct commits to main. All changes flow through agent-reviewed pull requests.
- **Synthetic-Data Rule.** Use only dummy data. Do not use Virtusa confidential or customer-sensitive data for this project.

4. Technology Stack

Choose any stack that matches your background. The stack does not affect evaluation.

Layer	Options
Frontend	React · Angular · Vue · server-rendered (Thymeleaf / Razor)
Backend	Java + Spring Boot · .NET + ASP.NET Core Web API · Python + FastAPI · Node.js + Express / NestJS
Database	In-memory (H2 / SQLite) · RDBMS (MySQL / MariaDB / SQL Server) · NoSQL (MongoDB / DynamoDB)
Build & Dependency Management	Maven / Gradle · dotnet CLI · pip + Poetry · npm / pnpm
Testing	JUnit / NUnit / pytest / Vitest · Playwright (E2E) · ArchUnit / dependency-cruiser / import-linter

Layer	Options
CI	GitLab CI (preferred for Virtusa) · GitHub Actions, with Claude Code Action integrated
MCP Server	Playwright MCP
Required Plugin	Claude Harness Engine plugin (Generator–Evaluator harness for autonomous long-running development)

5. Acceptance Criteria & Non-Functional Requirements

5.1 Functional Acceptance Criteria

ID	Criterion
AC-01	Customer can initiate a payment with beneficiary, amount, and narration; the system assigns a payment_id and accepts an idempotency_key; the initial state is PENDING
AC-02	Idempotency rule: re-submitting the same idempotency_key returns the original payment_id without creating a duplicate payment record
AC-03	Routing engine selects the rail based on amount and beneficiary type (e.g., RTGS for amounts ≥ ₹2,00,000; UPI for retail amounts ≤ ₹1,00,000; NEFT otherwise); the routing decision is logged with reason
AC-04	Payment lifecycle states PENDING → PROCESSING → SETTLED / FAILED / REFUNDED are enforced; invalid transitions raise InvalidPaymentStateException
AC-05	When the stubbed rail adapter returns a transient failure, the payment is retried per the configured retry policy; permanent failures move the payment to FAILED with a reason code
AC-06	Reconciliation matches outbound payment records against inbound settlement-file entries; unmatched entries are flagged in an exception report for ops review
AC-07	Daily settlement file is generated for all SETTLED payments of the day with matched / unmatched indicators; the generated file is append-only and immutable once written
AC-08	Customer can request a refund on a SETTLED payment; the refund creates a new reverse entry linked to the original; the original payment status moves to REFUNDED
AC-09	All state transitions and routing decisions are persisted as an immutable audit record with timestamp and actor
AC-10	Ops admin can view daily volumes by rail and status, the unmatched-entries queue, and the refund queue

Note. *Each AC must have at least one test referencing its AC-NN identifier.*

5.2 Non-Functional Requirements

ID	Requirement
NFR-01	Amount, fees, and refund values are computed in fixed-point (BigDecimal / decimal) — never floating-point
NFR-02	Payment records, state transitions, settlement entries, and refund records are all append-only
NFR-03	Beneficiary account numbers, IFSC codes, and payer details are masked in logs and never persisted in plaintext where avoidable
NFR-04	Authentication boundary at controller layer; customer / ops / admin roles are separated
NFR-05	Database migrations are append-only
NFR-06	Structured JSON logs with request correlation IDs and payment_id
NFR-07	Health endpoint returns 200 within 1 second of a successful startup
NFR-08	Architecture rules enforced as tests — duplicate payments for the same idempotency_key cannot be created; SETTLED payments cannot be mutated except via a REFUNDED transition

6. Functional Scope

6.1 Customer / End-User Functionality (User Journeys)

- **Initiate Payment.** Submit beneficiary, amount, and narration; receive a payment_id and an idempotency_key handshake.
- **Track Status.** View the payment lifecycle (PENDING → PROCESSING → SETTLED / FAILED / REFUNDED) and reason codes.
- **Request Refund.** Raise a refund on a SETTLED payment; track the reverse entry and refund status.
- **Payment History.** View past payments with timeline, rail used, and final status.

6.2 Internal & System Functionality

- **Routing Engine.** Rules-based rail selection (UPI / NEFT / RTGS / IMPS) by amount, currency, and beneficiary type.
- **Rail Adapters (stubbed).** Configurable adapters per rail with deterministic success / failure / transient-error responses for testing.
- **Idempotency Manager.** Enforces unique payment creation per idempotency_key across retries and duplicate submissions.
- **Retry & Backoff.** Configurable retry policy for transient failures; permanent failures are surfaced to ops.
- **Reconciliation Module.** Matches outbound payments against inbound settlement-file entries; surfaces unmatched items.
- **Settlement File Generator.** Daily batch job producing the append-only settlement file with matched / unmatched indicators.
- **Refund Engine.** Generates reverse entries linked to original payments; transitions states with audit.

- **Ops Dashboard.** Payment volumes by rail and status, unmatched queue, refund queue, retry pipeline.

6.3 Out of Scope

You do not need to build:

- Real UPI / NEFT / RTGS / IMPS network connections — all rails are stubbed adapters
- Real bank-to-bank messaging (SWIFT, etc.) — out of scope entirely
- Real clearing-house / RBI settlement infrastructure
- Real beneficiary verification or KYC checks (stub a flag)
- Foreign-exchange or international remittances
- Production deployment, secret management, multi-region, distributed locking

7. AI-Native Implementation Expectations

Foundation. Install the **Claude Harness Engine plugin** first. The harness ships 7 base agents (planner, generator, evaluator, security-reviewer, test-engineer, design-critic, ui-designer), 12+ hooks, base commands, sprint contracts, an evaluator wired to Playwright MCP, and a self-healing loop. On top of that foundation, your repository must add the project-specific substrate listed in this section. The rubric scores the final repository — both Harness-supplied and learner-added artifacts contribute, but the thresholds below are calibrated so that learners who only install Harness without customizing for the project domain will not meet them.

7.1 Claude Code Specs (40 marks)

- specs/app_spec.md (root spec) and specs/<feature>_spec.md per major feature, each with explicit Acceptance Criteria sections
- Layered CLAUDE.md — root + per-module + per-folder; AGENTS.md as a table of contents only
- At least 3 project-specific skills under .claude/skills/ added by you (examples: routing-rule-evaluator, idempotency-key-generator, reconciliation-matcher, settlement-file-builder, retry-policy-evaluator; or technical: state-machine-validator, archtest-author, audit-tracer)
- At least 2 project-specific custom commands under .claude/commands/ added by you (beyond Harness defaults)
- At least 2 project-specific hooks under .claude/hooks/ added by you (examples: idempotency-uniqueness-check.sh, payment-immutability-check.sh, mask-account-number-check.sh, amount-precision-check.sh)
- At least 3 project-specific agents under .claude/agents/ added by you. These may be **domain agents** (e.g., routing-agent, reconciliation-agent, refund-agent, settlement-file-agent), **technical agents** that strengthen the engineering substrate (e.g., state-machine-validator-agent, idempotency-validator-agent, archtest-author-agent), or a mix of both. You may rename Harness's generator → implementer, evaluator → reviewer, test-engineer → tester to align with canonical naming
- At least one programmatic Claude Agent SDK usage in scripts/
- Your own plugin.json packaging your project's substrate (in addition to Harness's plugin.json)
- At least one project-specific debugging-log or post-mortem document under docs/ evidencing environment-first resolution

- Playwright MCP configured in .mcp.json and actively used by the evaluator

7.2 Business Understanding (20 marks)

- docs/business-case.md covering: problem, target users, success metrics, domain rules, value proposition. This is the document the reviewer plugin reads — see Section 8
- At least 4 rule / policy / validator files under src/domain/ (or equivalent) implementing payments-operations business rules
- Frontend evidence (any of: React / Angular / Vue) with at least one responsive layout
- Acceptance criteria expressed in testable form — Given-When-Then or AC-NN identifiers — for at least 4 specs

7.3 Architectural Discipline (8 marks)

- Architecture-test directory (tests/architecture/ or equivalent) with at least 3 structural tests
- ArchUnit / dependency-cruiser / import-linter configured to enforce layering rules
- TDD discipline document at docs/tdd.md and at least 10 test files committed across history showing red–green–refactor pattern

7.4 Technical Implementation (26 marks)

- docs/architecture.md with at least 1 architecture diagram (Mermaid or image) and an explicit layered structure (controllers / services / repositories / domain)
- Substantive codebase with at least 3000 lines of meaningful generated code
- Playwright (or equivalent) for UI validation with at least one snapshot directory
- Tests tagged with AC IDs — every spec AC has at least 1 corresponding test
- At least 20 unit tests with a coverage artefact (coverage.xml, lcov.info, or equivalent) and the coverage tool declared in dependencies

7.5 CI/CD (6 marks)

- .gitlab-ci.yml (or .github/workflows/) with the build and test pipeline
- Claude Code Action (or equivalent) wired into the CI workflow
- At least 3 PR-driven merges (use git merge --no-ff to preserve PR history); zero direct commits to main

8. Expected Outcomes & Deliverables

By the end of 20 hours, the submitted Git repository must contain the Mandatory items below. Good-to-Have items differentiate Merit and Distinction submissions.

8.1 Mandatory

1. **Working Application.** Payments hub deployable locally via a single command (./mvnw spring-boot:run, npm start, dotnet run, or equivalent) with seed data (rail adapters, sample beneficiaries) and a README quick-start.
2. **docs/business-case.md.** The business context document the reviewer plugin reads. Use this brief's Sections 3 and 6 as the source; rewrite in your own words covering problem, users, success metrics, domain rules, and value proposition.

3. **Specifications.** specs/app_spec.md plus per-feature specs (initiation, routing, idempotency, rail-adapters, reconciliation, settlement, refund) with explicit Acceptance Criteria sections.
4. **Layered CLAUDE.md Hierarchy.** Root + per-module + per-folder; AGENTS.md as a table of contents.
5. **Project Agent Substrate.** .claude/agents/, .claude/skills/, .claude/commands/, .claude/hooks/ — Harness foundation plus your project-specific additions as listed in Section 7.1.
6. **Claude Harness Engine Plugin Integration.** Plugin installed; project scaffolded via the harness; at least one full sprint cycle (Generator → Evaluator → ratcheting → PR) driven through it. Sprint contracts and evaluator outputs committed under sprint-contracts/ and specs/reviews/.
7. **Plugin Manifest.** Your own plugin.json at the repository root packaging your payments-operations substrate; .mcp.json with Playwright MCP active.
8. **Test Suite.** Unit tests (incl. idempotency edge cases, routing rules, reconciliation matching, refund state transitions), architecture tests, AC-tagged tests, Playwright UI tests, coverage report — all committed.
9. **CI Pipeline.** .gitlab-ci.yml (or equivalent) with Claude Code Action wired in; passing build on main.
10. **PR-Driven Git History.** At least 3 PR-driven merges; zero direct pushes to main.

8.2 Good-to-Have

11. **Architecture & TDD Documentation.** docs/architecture.md with at least one diagram (C4 context or sequence diagram for the payment routing and reconciliation flow); docs/tdd.md describing the red–green–refactor approach with one worked example (e.g., AC-02 idempotency replay).
12. **Autonomous Fix Loop Evidence.** At least one recorded loop where an agent detected a failure (e.g., a duplicate-payment bug exposed by an idempotency replay), reproduced, fixed, validated, and opened a PR. Capture the trace under docs/fix-loops/.
13. **Knowledge Deposit Log.** docs/knowledge-deposits.md recording recurring mistakes encoded back into rules, hooks, or skills (e.g., a hook that blocks any code path bypassing the idempotency check on payment creation).
14. **Multiple Sprint Cycles.** More than one harness-driven sprint covering: initiation + routing → rail adapters + retry → reconciliation + settlement → refunds + ops dashboard.
15. **Bonus Agents.** Beyond the required additions: janitor, performance-auditor, doc-writer, or other domain-specific or technical agents.

On completion of the capstone, you will have demonstrated the skill the industry is seeking: building an idempotent, multi-rail payments hub with routing, reconciliation, and settlement workflows — without writing the code yourself, while maintaining full control of quality, security, and architecture through AI-native engineering practices.